

## 1. Basic Syntax

### Variable Declaration

```
var name = 'Johnny'; // Inferred type: String
String anotherName = 'Arjun'; // Explicit type
int age = 30;
```

### Final and Const

```
final height = 6.2; // Runtime constant
const PI = 3.14;    // Compile-time constant
```

## 2. Data Types

- **Numbers:** `int`, `double`
- **Strings:** `String`
- **Boolean:** `bool`
- **Lists (Arrays):** `List<int> nums = [1, 2, 3];`
- **Maps (Dictionaries in Python):** `Map<String, int> scores = {'Arjun': 90, 'Bablu': 85};`

## 3. Functions

### Basic Function

```
void printMessage(String message) {
    print(message);
}
```

```
String greet(String name) => 'Hello, $name'; // Short syntax
```

### Optional Parameters (Named and Positional):

```
void displayInfo(String name, {int age = 0, String? city}) {
    print('$name, $age, $city');
}
```

```
displayInfo('Johnny', age: 25, city: 'NY');
```

## 4. Control Flow

### If-Else:

```
if (age > 18) {
    print('Adult');
} else {
    print('Minor');
}
```

### For Loop:

```
for (int i = 0; i < 5; i++) {
    print(i);
}
```

### For Each Loop (similar to JS `for...of`):

```
List<String> fruits = ['Apple', 'Banana', 'Mango'];
```

```
for (var fruit in fruits) {  
    print(fruit);  
}
```

#### **Switch-Case:**

```
switch (day) {  
    case 'Monday':  
        print('Start of the week');  
        break;  
    default:  
        print('Invalid day');  
}
```

## **5. Classes and Objects**

#### **Class Definition:**

```
class Person {  
    String name;  
    int age;  
  
    // Constructor  
    Person(this.name, this.age);  
  
    // Named Constructor  
    Person.withoutAge(this.name) : age = 0;  
  
    // Method  
    void greet() {  
        print('Hello, $name');  
    }  
}  
  
var p = Person('Arjun', 25);  
p.greet();
```

#### **Inheritance:**

```
class Employee extends Person {  
    String job;  
  
    Employee(String name, int age, this.job) : super(name, age);  
  
    @override  
    void greet() {  
        super.greet();  
        print('I am an $job');  
    }  
}
```

```
}
```

#### **Abstract Class:**

```
abstract class Shape {  
    void draw();  
}
```

```
class Circle extends Shape {  
    @override  
    void draw() {  
        print('Drawing Circle');  
    }  
}
```

## **6. Exception Handling**

#### **Try-Catch:**

```
try {  
    var result = 10 ~/ 0; // Integer division by zero  
} catch (e) {  
    print('Error: $e');  
} finally {  
    print('Cleanup if necessary');  
}
```

## **7. Asynchronous Programming**

#### **Future (Similar to Promises in JS):**

```
Future<String> fetchData() async {  
    return 'Data loaded';  
}
```

```
fetchData().then((data) => print(data));
```

#### **Async-Await:**

```
Future<void> fetchUserOrder() async {  
    var order = await getOrder();  
    print('Order received: $order');  
}
```

## **8. Collections (Lists, Sets, Maps)**

#### **List (Array in JS):**

```
List<int> numbers = [1, 2, 3];  
numbers.add(4);
```

#### **Set (Unique elements):**

```
Set<String> colors = {'red', 'green', 'blue'};
```

```
colors.add('red'); // No duplicate
```

### Map (Similar to Python dict):

```
Map<String, int> ages = {  
    'Johnny': 25,  
    'Arjun': 30,  
};
```

## 9. Null Safety

Dart uses **null safety** to avoid null exceptions at runtime.

```
String? nullableString;  
nullableString = null; // Allowed
```

Use **!** to assert that a value is non-null:

```
String? name;  
print(name!.length); // Throws an error if name is null
```

## 10. Constructors and Named Parameters

**Constructor with Named Parameters:**

```
class Car {  
    String brand;  
    int year;  
  
    Car({required this.brand, this.year = 2020});  
}
```

```
var car = Car(brand: 'Tesla', year: 2022);
```

## 11. Lambda Expressions

**Short Functions (Arrow Syntax):**

```
var add = (int a, int b) => a + b;
```

## 12. Dart Collections and Loops

**Collection Operations:**

```
var list = [1, 2, 3, 4];  
var doubledList = list.map((e) => e * 2).toList();
```

## 13. Getters and Setters

**Custom Getters and Setters:**

```
class Rectangle {  
    int width, height;  
  
    Rectangle(this.width, this.height);  
  
    int get area => width * height; // Getter
```

```
    set width(int w) => this.width = w;  // Setter  
}
```